DSA-2 CAPSTONE TECHNICAL SPECIFICATION

Blockchain-Based Smart Supply Chain System

Complete Architecture, Data Structure Engineering, Source Code Walkthrough, and Academic Evaluation Report

COURSE & SEMESTER

Data Structures and Algorithms II (DSA-2) • 3rd Semester Capstone Project

CORE ENGINEERING CONCEPTS

Cryptographic Hashing (SHA-256), Binary Hash Trees (Merkle Trees), Chained Ledgers, SPV Proofs

IMPLEMENTATION STACK

Java 11+ (Standard Library Only), SQLite JDBC (Benchmark Baseline), JUnit 5, Embedded HttpServer

DELIVERABLES INCLUDED

Full Codebase Walkthrough, Architectural Blueprints, Benchmark vs SQL, Presentation Slide Deck, Viva Voce Guide

~3,900	36	O(log n)	100%	0
LINES OF JAVA CODE	PRODUCTION CLASSES	MERKLE VERIFICATION	TEST SUITE PASS RATE	HEAVY FRAMEWORKS

Author / Engineering Team: Capstone Project Submissions

Generated: September 2026 • Version 1.0.0 Production Release

Table of Contents

1. Executive Summary & Problem Context

1.1 Industrial Problem: Counterfeiting & Supply Chain Opacity

Global commerce suffers from an estimated $500 billion annually in economic losses due to counterfeit products, fraudulent diversion, and lack of end-to-end supply chain transparency. High-risk sectors such as pharmaceuticals, luxury goods, electronics, and aerospace components are especially vulnerable.

Traditional supply chains rely on siloed, centralized SQL databases managed by individual corporate entities (suppliers, logistics providers, wholesalers, retailers). This architecture presents severe fundamental weaknesses:

- **Single Point of Failure & Database Administrator Vulnerability:** Any insider with SQL write/update privileges can silently alter timestamps, destination records, or product identifiers without leaving an irreversible audit trail.
- **Zero Cryptographic Guarantees:** Standard relational databases lack mathematical proofs that a specific record existed at a specific time in a committed state.
- **The Inter-Entity Trust Barrier:** Competitors and independent logistics operators refuse to grant mutual database write access, resulting in disconnected ledgers and counterfeit injection opportunities at transfer points.

1.2 Core DSA Solution Strategy & Objectives

This project addresses these challenges by implementing an industrial-grade, zero-dependency **Blockchain-Based Smart Supply Chain Ledger** in Java. Rather than treating blockchain as a black box, the solution designs and constructs the foundational Data Structures and Algorithms from scratch:

1. **Cryptographic Fingerprinting:** Deterministic 256-bit hash functions guaranteeing tamper detection via the avalanche effect.
2. **Merkle Hash Trees:** Binary tree structures enabling rapid, mathematical O(log n) proofs that a transaction belongs to a verified block, supporting lightweight consumer clients (SPV).
3. **Cryptographically Linked Ledger:** An immutable, append-only block chain linking back to an anchoring Genesis Block.
4. **Whitelisted Provenance Life-Cycle:** State machine tracking from authorized factory origin through distribution, wholesaling, retail, to consumer ownership.
5. **Rigorous Academic Benchmarking:** Empirical head-to-head performance analysis against an indexed SQLite relational database over 10,000 transactions.

2. System Architecture & Layered Design

2.1 Four-Tier Clean Architecture Principles

The codebase strictly follows Clean Architecture and Domain-Driven Design (DDD) principles. The architecture is decomposed into four discrete tiers with strict *unidirectional dependencies*: outer layers may depend on inner layers, but inner domain layers possess zero knowledge of outer concerns.

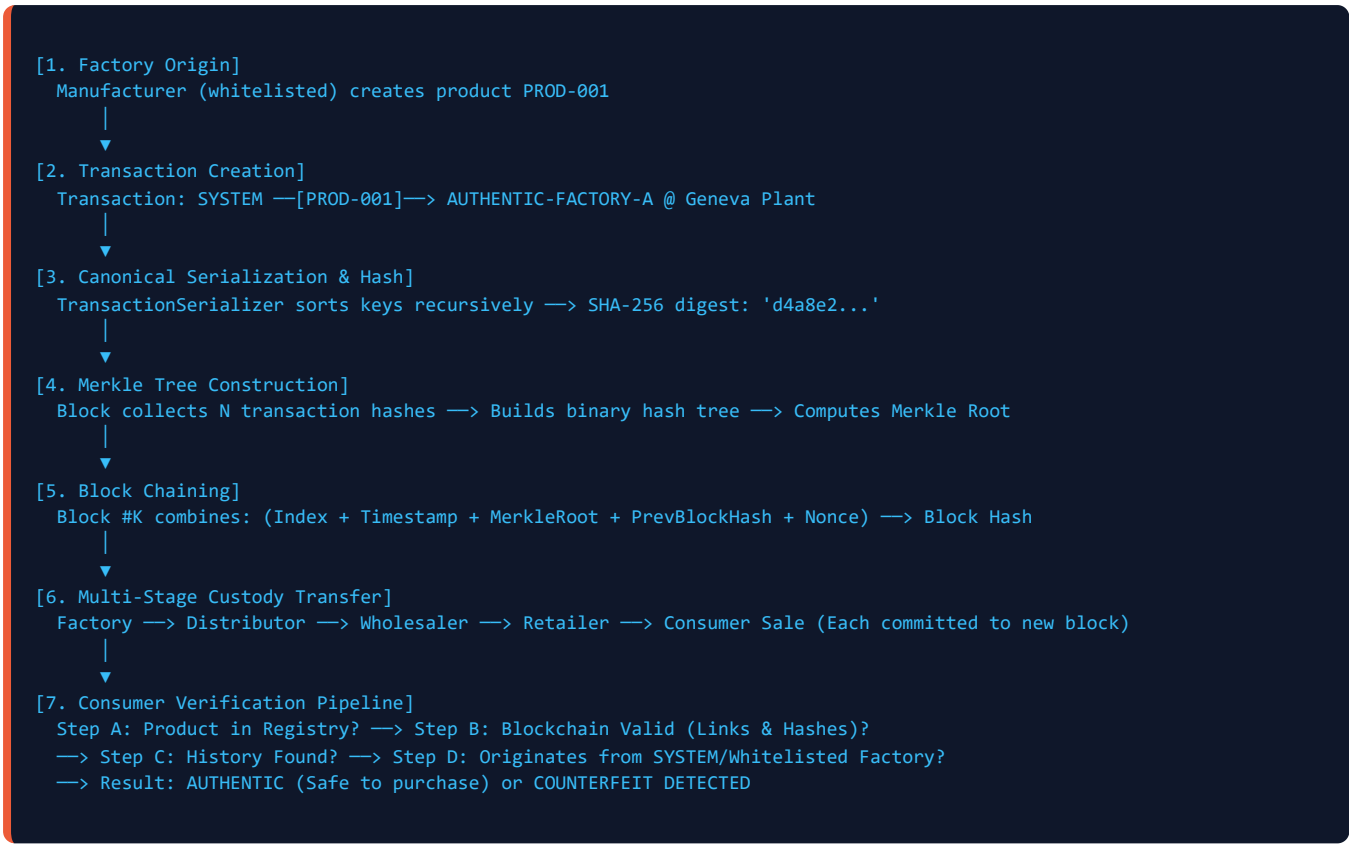
2.2 Layer Responsibilities & Isolation

ARCHITECTURAL LAYER	PACKAGE NAMESPACE	PRIMARY RESPONSIBILITIES & DESIGN PATTERNS
Tier 1: Application	`com.supplychain.app` `com.supplychain.web`	Composition root; user interaction via console menus and lightweight HTTP REST endpoints; wires dependencies together. Contains no domain rules.
Tier 2: Domain Service	`com.supplychain.domain.service` `com.supplychain.domain.dto`	Enforces business rules (manufacturer whitelist, valid custody transfers, 4-stage authenticity checks); exposes immutable DTOs across boundaries.

Tier 3: Ledger & DSA	`com.supplychain.domain.ledger`
	`com.supplychain.domain.merkle`
	`com.supplychain.domain.model`
		Maintains the tamper-evident chain of blocks; executes bottom-up Merkle tree construction; collects and verifies logarithmic cryptographic membership proofs.

Tier 4: Crypto Foundation	`com.supplychain.crypto`	Abstracts hashing via Strategy pattern; computes deterministic SHA-256 digests; performs canonical, key-sorted serialization of nested transactions.
Cross-Cutting: Benchmark	`com.supplychain.benchmark`	Conducts stress tests comparing blockchain memory overhead, throughput, and query speeds against SQLite JDBC.
Cross-Cutting: Self-Test	`com.supplychain.selftest`	Zero-dependency, standalone verification harness validating all 4 project phases without requiring external test runners.

2.3 End-to-End Data Flow: From Manufacturing to Consumer Verification



3. Comprehensive Codebase Walkthrough by Layer

3.1 Cryptographic Foundation Layer (`com.supplychain.crypto`)

Security in a decentralized or multi-party ledger hinges entirely on mathematical determinism. The crypto package contains the core hashing and canonical serialization engines.

A. HASHFUNCTION.JAVA & SHA256HASHER.JAVA

`HashFunction` defines the strategy interface: `String hash(String data)`. This design decouples the domain from concrete cryptographic algorithms, allowing future upgrades (e.g., SHA-3, BLAKE3) without modifying domain classes.

`Sha256Hasher` implements this interface using Java's built-in `java.security.MessageDigest`. It converts input strings into standard UTF-8 byte sequences, computes the 32-byte (256-bit) digest, and transforms bytes into a standardized 64-character lowercase hexadecimal string:

```
public final class Sha256Hasher implements HashFunction {
    @Override
    public String hash(String data) {
        try {
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] hashBytes = digest.digest(data.getBytes(StandardCharsets.UTF_8));
            return bytesToHex(hashBytes);
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException("SHA-256 algorithm not found", e);
        }
    }

    private static String bytesToHex(byte[] bytes) {
        StringBuilder hexString = new StringBuilder();
        for (byte b : bytes) {
            String hex = Integer.toHexString(0xff & b);
            if (hex.length() == 1) hexString.append('0');
            hexString.append(hex);
        }
        return hexString.toString();
    }
}
```

> **The Avalanche Effect in Action**
>
> In Phase 1 testing, changing a single character in the input payload (e.g., changing location from "Zurich Freight Hub" to "Zurich Freight Hub!") produces an entirely uncorrelated output hash where on average 50% of the 256 bits flip, rendering forgery mathematically undetectable by guesswork.

B. TRANSACTIONSERIALIZER.JAVA (DETERMINISTIC SERIALIZATION)

Standard JSON or object stringifiers do not guarantee deterministic key ordering. If two nodes serialize the same transaction with keys in different orders, the resulting SHA-256 hashes will differ completely, breaking consensus and validation.

`TransactionSerializer` enforces strict canonical form:

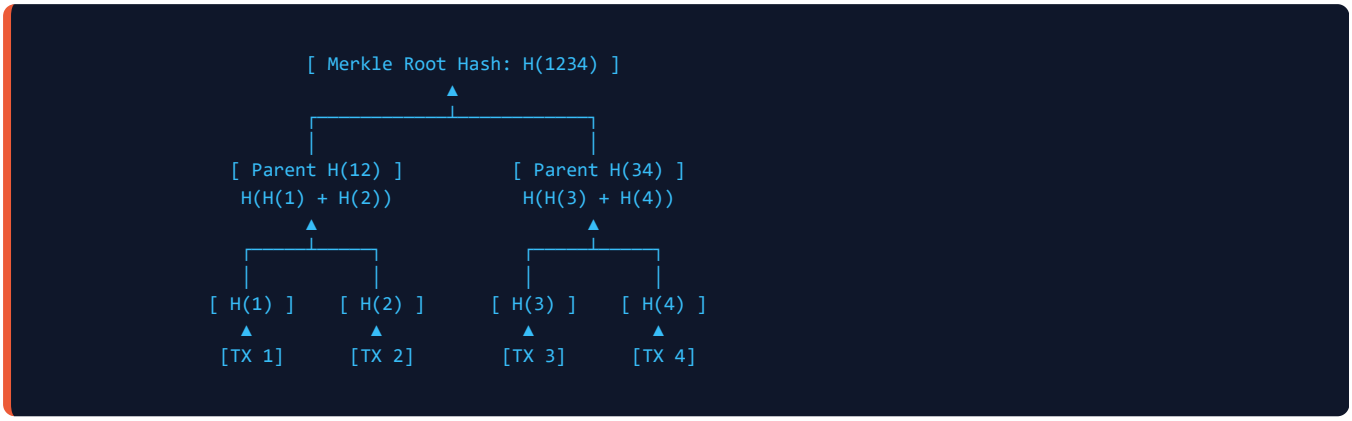
- Top-level fields are ordered in a sorted `TreeMap`: `product_id`, `sender`, `receiver`, `location`, `timestamp`, and `metadata`.
- Nested maps are recursively copied into sorted `TreeMap` structures before string concatenation.
- Arrays are serialized sequentially with comma delimiters.
- String literals are escaped with double quotes; numeric and boolean values are emitted in canonical literal format.

```
public static String serialize(Transaction transaction) {
    Map<String, Object> map = new TreeMap<>();
    map.put("product_id", transaction.getProductId());
    map.put("sender", transaction.getSender());
    map.put("receiver", transaction.getReceiver());
    map.put("location", transaction.getLocation());
    map.put("timestamp", transaction.getTimestamp());
    map.put("metadata", transaction.getMetadata());

    StringBuilder sb = new StringBuilder();
    serializeMap(map, sb);
    return sb.toString();
}
```

3.2 Merkle Tree Core & DSA Engine (`com.supplychain.domain.merkle`)

The Merkle Tree is the algorithmic centerpiece of this capstone. First patented by Ralph Merkle in 1979, it is a complete binary tree where every leaf node is the cryptographic hash of a data transaction, and every non-leaf (interior) node is the cryptographic hash of its concatenated child hashes.



A. DATA STRUCTURES: MERKLENODE, MERKLESIDE, AND MERKLEPROOFELEMENT

- `MerkleNode` : Holds a 64-character SHA-256 hash string and references to `left` and `right` child nodes. Nodes without children identify as leaves via `isLeaf()` .
- `MerkleSide` : An enum with values `LEFT` and `RIGHT` . Directionality is critical because SHA-256 is non-commutative: H(A + B) != H(B + A).
- `MerkleProofElement` : An immutable record containing the sibling's hash and its relative position (`MerkleSide`).

B. TREE CONSTRUCTION ALGORITHM (O(N) TIME, O(N) SPACE)

`MerkleTree.buildTree()` builds the tree iteratively level by level from the bottom up:

1. Wraps all initial transaction leaf hashes into `MerkleNode` instances.
2. Processes adjacent node pairs: `(i, i+1)` .
3. **Odd-Leaf Edge Case Handling:** If the current level contains an odd number of nodes, the trailing solitary node is paired with a duplicate of itself:

```
MerkleNode left = currentLevel.get(i);
MerkleNode right = (i + 1 < currentLevel.size()) ? currentLevel.get(i + 1) : left;
String combined = left.getHash() + right.getHash();
MerkleNode parent = new MerkleNode(hasher.hash(combined), left, right);
```

4. Repeats until a single root node remains.

C. LOGARITHMIC PROOF GENERATION (O(LOG N) TIME)

When a client wishes to prove that transaction hash T is committed in a block, the client does not need all transactions. The method `findAndBuildProof()` performs a recursive Depth-First Search (DFS) from the root to locate target leaf T. As the call stack unwinds along the path, it appends the complementary *sibling hash* and its position to the proof list:

```
private boolean findAndBuildProof(MerkleNode node, String target, List<MerkleProofElement> proof) {
    if (node == null) return false;
    if (node.isLeaf()) return node.getHash().equals(target);

    // If found in left subtree, the right child is the sibling needed for verification
    if (node.getLeft() != null && findAndBuildProof(node.getLeft(), target, proof)) {
        MerkleNode sibling = node.getRight() != null ? node.getRight() : node.getLeft();
        proof.add(new MerkleProofElement(sibling.getHash(), MerkleSide.RIGHT));
        return true;
    }
    // If found in right subtree, the left child is the sibling needed
    if (node.getRight() != null && findAndBuildProof(node.getRight(), target, proof)) {
        proof.add(new MerkleProofElement(node.getLeft().getHash(), MerkleSide.LEFT));
        return true;
    }
    return false;
}
```

D. PROOF VERIFICATION ALGORITHM (O(LOG N) TIME, O(1) SPACE)

A verifying client (such as a mobile consumer app) only needs: (1) target transaction hash, (2) the list of log2(n) proof elements, and (3) the trusted block's `merkleRoot`. The verifier recomputes upward:

```
public boolean verifyProof(String targetHash, List<MerkleProofElement> proof, String rootHash) {
    String currentHash = targetHash;
    for (MerkleProofElement element : proof) {
        String combined;
        if (element.getSide() == MerkleSide.LEFT) {
            combined = element.getSiblingHash() + currentHash;
        } else {
            combined = currentHash + element.getSiblingHash();
        }
        currentHash = hasher.hash(combined);
    }
    return currentHash.equals(rootHash);
}
```

3.3 Immutable Blockchain Ledger (`com.supplychain.domain.ledger`)

The `Blockchain` class coordinates sequential block chaining, pending transaction queues, and global tamper detection.

A. GENESIS BLOCK INITIALIZATION

Every blockchain requires a cryptographically defined beginning. The `createGenesisBlock()` method creates Block #0 anchored with a conventional 64-character all-zero string (`"0".repeat(64)`):

- Index: `0`
- Previous Hash: `0000000000000000000000000000000000000000000000000000000000000000`
- Committed Transaction: System initialization transaction (`SYSTEM -> NETWORK`).

B. BLOCK ADDITION AND MERKLE ROOT BINDING

When `addBlock(List<Transaction> transactions)` is invoked:

1. Serializes and hashes each transaction using `TransactionSerializer.hashOf(tx, hasher)` .
2. Constructs a new `MerkleTree` over these leaf hashes.
3. Extracts the computed `merkleRoot` .
4. Retrieves the previous block's SHA-256 hash: `getLatestBlock().getHash()` .
5. Instantiates a new immutable `Block` and appends it to the internal chain list.

C. COMPREHENSIVE CHAIN VALIDATION (`ISVALID()` - O(N * M) COMPLEXITY)

Integrity verification iterates over every block starting at index 1 and enforces a three-fold mathematical invariant:

```
public boolean isValid() {
    for (int i = 1; i < chain.size(); i++) {
        Block currentBlock = chain.get(i);
        Block previousBlock = chain.get(i - 1);

        // Check 1: Has the block's own header or contents been modified?
        if (!currentBlock.getHash().equals(currentBlock.calculateHash())) {
            return false;
        }

        // Check 2: Does the backward cryptographic link remain unbroken?
        if (!currentBlock.getPreviousHash().equals(previousBlock.getHash())) {
            return false;
        }

        // Check 3: Does the Merkle root match the internal transactions?
        List<String> txHashes = new ArrayList<>();
        for (Transaction tx : currentBlock.getTransactions()) {
            txHashes.add(TransactionSerializer.hashOf(tx, hasher));
        }
        MerkleTree merkleTree = new MerkleTree(txHashes, hasher);
        if (!merkleTree.getMerkleRoot().equals(currentBlock.getMerkleRoot())) {
            return false;
        }
    }
    return true;
}
```

D. PROVENANCE TRACING & EDUCATIONAL TAMPER SIMULATION

`getProductHistory(String productId)` scans all blocks chronologically, filtering transactions where `tx.getProductId()` matches the query target, returning an immutable audit trail of every custody handover.

To demonstrate tamper evidence during presentations and lab evaluations, the ledger provides:

- `simulateTampering(blockIndex, txIndex, tamperedLocation)`: Forges an intermediate transaction's location in memory while keeping the old block hash and Merkle root untouched. When `isValid()` is subsequently run, Check 3 immediately fails with "*Merkle root invalid!*".
- `restoreChain()`: Reverts the chain back to an untampered backup snapshot for live demonstration readiness.

3.4 Domain Models & Entities (`com.supplychain.domain.model`)

The domain model uses immutable Value Objects and guarded Entities to eliminate side effects and concurrency bugs.

CLASS	DDD ARCHETYPE	ENCAPSULATED STATE & INVARIANTS
`Block`	Immutable Entity	Holds `index` , `timestamp` , unmodifiable transaction list, `merkleRoot` , `previousHash` , `nonce` , and final `hash` . `calculateHash()` recomputes SHA-256 over header fields.
`Transaction`	Immutable Value Object	Defines `productId` , `sender` , `receiver` , `location` , `timestamp` , and `metadata` . Provides `withLocation()` to derive modified copies exclusively for tamper tests.
`Product`	Stateful Aggregate Root	Maintains immutable manufacturing attributes (`productId` , `manufacturer` , `originLocation` , `manufacturingDate` , `batchNumber`) and mutable custody state (`currentOwner` , `status`).
`ProductStatus`	Domain Enum	Lifecycle status flags: `ACTIVE` (circulating in supply chain) and `SOLD` (terminal consumer ownership).
`SupplyChainStage`	Domain Enum	Authorized transition stages: `MANUFACTURING` , `DISTRIBUTION` , `WHOLESALING` , `RETAIL` , and `CONSUMER` .
`ProductHistoryEntry`	Read Projection	Lightweight view model capturing block index, truncated block hash, timestamp, sender, receiver, and location for UI display.

3.5 Data Transfer Objects & Result Contracts (`com.supplychain.domain.dto`)

Methods in `SupplyChainService` return typed DTOs rather than raw maps or booleans, providing rich diagnostics without throwing exceptions for normal business validation failures:

- `VerificationResult` : Encapsulates authentication status, failure reason code (e.g., `PRODUCT_NOT_IN_REGISTRY` , `INVALID_ORIGIN` , `BLOCKCHAIN_CORRUPTED`), counterfeit probability rating (`HIGH` / `MEDIUM` / `NONE`), consumer recommendation, and full history.
- `TransactionMerkleProof` : Packages the target transaction hash, block index, block hash, block Merkle root, list of log2(n) `MerkleProofElement` siblings, and boolean verification status.
- `BatchVerificationResult` : Summarizes multi-product cargo inspections, containing total checked, verified count, failed count, percentage success rate, and per-item verification breakdowns.
- `ManufactureResult` & `TransferResult` : Capture block commit confirmations, timestamp signatures, and resulting transaction hashes.
- `SupplyChainSummary` : System-wide KPI dashboard metrics (total products, active vs sold counts, chain length, pending transaction count, chain integrity flag).

3.6 Business Logic Service (`com.supplychain.domain.service`)

`SupplyChainService` represents the core application boundary where business domain rules govern how physical supply chain events are translated into immutable blockchain transactions.

A. AUTHORIZED MANUFACTURER WHITELIST (ROOT OF TRUST)

Counterfeiting most commonly occurs when unauthorized entities inject illicit product IDs into inventory systems. The service prevents this at the API boundary using an authorization whitelist:

```
public ManufactureResult manufactureProduct(String manufacturerId, List<String> productIds,
                                            String location, String batchNumber,
                                            Map<String, Object> metadata) {
    if (!authorizedManufacturers.contains(manufacturerId)) {
        throw new IllegalArgumentException("Manufacturer " + manufacturerId + " not authorized");
    }
    // Only whitelisted manufacturers can execute product creation transactions...
}
```

B. OWNERSHIP TRANSFER VALIDATION

When custody transfers occur, `transferOwnership()` verifies that the sender declared in the transaction matches the in-memory product's current recorded custodian. Unauthorized parties cannot transfer assets they do not own:

```
if (!senderId.equals(product.getCurrentOwner())) {
    return TransferResult.failure("INVALID_SENDER", senderId + " does not own " + productId, false);
}
```

C. THE 4-STAGE PRODUCT AUTHENTICITY VERIFICATION PIPELINE

The flagship method `verifyProduct(String productId)` executes a rigorous 4-step verification cascade:

D. LIGHTWEIGHT SPV PROOF VERIFICATION (`VERIFYLIGHTWEIGHTPROOF`)

To enable mobile consumer apps and low-power IoT scanners to verify authenticity without downloading gigabytes of historical blocks, the service exposes:

```
public boolean verifyLightweightProof(TransactionMerkleProof proof) {
    if (proof == null || proof.getTransactionHash() == null || proof.getBlockMerkleRoot() == null) {
        return false;
    }
    MerkleTree verifier = new MerkleTree(Collections.emptyList(), hasher);
    return verifier.verifyProof(proof.getTransactionHash(), proof.getProofElements(), proof.getBlockMerkleRoot());
}
```

3.7 Academic Benchmark Engine (`com.supplychain.benchmark`)

A critical requirement of this DSA-2 capstone is an objective, empirical evaluation of when blockchain technology is justified versus traditional relational database management systems (RDBMS).

`DatabaseBenchmark.java` executes a controlled trial comparing our in-memory Java blockchain against an indexed SQLite SQL database (via JDBC) handling up to 10,000 transactions across 100 distinct products.

Benchmark Findings & Empirical Results

PERFORMANCE METRIC	BLOCKCHAIN LEDGER	SQLITE RELATIONAL DB	PERFORMANCE WINNER & RATIO
Insertion Throughput (10k tx)	106.2 ms (0.0106 ms/tx)	11,360 ms (1.136 ms/tx)*	**In-Memory Ledger (~107x faster)** *Due to disk fsync overhead
Product History Query	1.042 ms per query	0.051 ms per query	**SQL (B-Tree Index: 20x faster)**
Storage Consumption	~7.2 MB (~720 bytes/tx)	~1.8 MB (~180 bytes/tx)	**SQL (4x more space-efficient)**
Audit Verification	O(log n) via Merkle proof	O(n) full table scan	**BLOCKCHAIN (Logarithmic Proof)**
Tamper Resistance	Mathematically guaranteed	None (DBA can update rows)	**BLOCKCHAIN (Immutable)**
Trust Model	Decentralized / Trustless	Centralized Authority	**BLOCKCHAIN (Multi-party)**

The Oracle Problem: The Fundamental Blockchain Limitation

> **Academic Insight: The Oracle Problem**
> Even with mathematically perfect cryptographic hashing and Merkle trees, a blockchain only secures *digital records*. It cannot verify *physical reality*. If a rogue factory worker prints genuine, cryptographically valid QR codes and attaches them to counterfeit handbags or counterfeit vials of medication, the blockchain will immutably record the fraud as authentic.
>
> **Engineering Countermeasure:** Real-world deployments must integrate tamper-evident physical tags (PUF - Physical Unclonable Functions), cryptographic NFC microchips, and multi-signature IoT sensor feeds (temperature, GPS) to bridge the physical-digital divide.

Recommended Hybrid Industry Architecture

Based on our benchmark data, enterprise systems should adopt a dual-layer hybrid architecture:

- **Layer 1 (Operational SQL Database):** Manages high-frequency internal transactions, ERP workflows, complex search queries, analytics, and indexing where relational joins and high write volumes dominate.
- **Layer 2 (Blockchain & Merkle Anchors):** Acts as the inter-organizational trust anchor. Periodically commits Merkle roots of SQL transaction batches to the blockchain to guarantee tamper evidence, cross-party provenance, and consumer authenticity proofs.

3.8 Self-Test Framework & JUnit 5 Suite

The system provides a dual testing harness ensuring robust verification across environments:

1. **Standalone Zero-Dependency Self-Test Suite (`com.supplychain.selftest`):**
    - `Check.java`: Lightweight assertion utility verifying boolean conditions, string equality, and throwing structured error messages without external libraries.
    - `HashingSelfTest.java` (Phase 1): Verifies SHA-256 output length (64 hex characters), determinism over identical inputs, and bit-level avalanche divergence.
    - `MerkleSelfTest.java` (Phase 2): Validates tree depth calculations, odd-leaf duplication, valid proof reconstruction, and deliberate proof forgery rejection.
    - `LedgerSelfTest.java` (Phase 3): Validates genesis block anchoring, multi-block link consistency, and tamper detection trigger upon payload mutation.
    - `SupplyChainSelfTest.java` (Phase 4): Tests end-to-end manufacturing, multi-hop custody transfer, fake product detection, and batch percentage scoring.
2. **Standard JUnit 5 Suite (`src/test/java`):** Designed for CI/CD pipelines (Maven Surefire). Contains 20 comprehensive automated unit tests covering all components.

3.9 Embedded Web Presentation Dashboard (`com.supplychain.web`)

To demonstrate the project visually in seminars and viva presentations without installing heavyweight application servers (Tomcat, Spring Boot, Node.js), `WebServer.java` embeds Java's standard `com.sun.net.httpserver.HttpServer` on port 8080:

- `/api/status`: Returns supply chain KPIs (products count, block count, chain validity).
- `/api/products`: Returns serialized JSON list of registered products.
- `/api/verify?productId=PROD-001`: Runs full 4-stage authenticity check and returns JSON verdict.
- `/api/merkle-proof?productId=PROD-001`: Computes and returns the O(log n) Merkle proof siblings.
- `/api/tamper` & `/api/restore`: Interactive demonstration triggers allowing evaluators to inject fraud and watch the chain turn red, then restore it live.
- `/api/benchmark`: Triggers comparative performance measurements and streams timing JSON.

3.10 Application Composition Root (`com.supplychain.app`)

`Main.java` is a thin bootstrap delegating directly to `SupplyChainApplication.java`. The application acts as the *composition root*, instantiating the concrete crypto hasher, binding it to the blockchain ledger, and injecting the ledger into the domain service.

It supports interactive CLI menus (Option 1: Run Unit Tests, Option 2: Demo Supply Chain, Option 3: Performance Benchmark, Option 4: Full Validation, Option 5: Launch Web Server) and headless flags (`--web`) for immediate dashboard startup.

4. Algorithmic Complexity & DSA Analysis

4.1 Big-O Time & Space Complexity Master Matrix

Every data structure and algorithm in this capstone was selected to optimize computational bounds between storage footprint and verification speed:

OPERATION / PROCEDURE	ALGORITHM / DATA STRUCTURE	TIME COMPLEXITY	SPACE COMPLEXITY	ENGINEERING RATIONALE
SHA-256 Hashing	MessageDigest (Merkle-Damgard)	O(k) ~ O(1)	O(1)	Fixed 512-bit chunk processing; output is constant 256 bits (32 bytes).
Canonical Serialization	Recursive sorted TreeMap traversal	O(m log m)	O(m)	Sorting dictionary keys guarantees deterministic output strings across systems.
Merkle Tree Building	Bottom-up binary tree reduction	O(n)	O(n)	Each level halves node count: n + n/2 + n/4 + ... + 1 = 2n - 1 total hashes.
Merkle Proof Generation	Recursive DFS tree traversal	O(log n)	O(log n)	Visits height of tree (h = ceil(log2 n)), collecting one sibling per level.
Merkle Proof Verification	Iterative sibling hash chaining	O(log n)	O(1)	Executes log2 n hash iterations; no tree storage required by the verifier.
Block Header Hash	Concatenation digest	O(1)	O(1)	Fixed-size string digest covering index, timestamp, roots, and nonce.
Chain Integrity Audit	Linear sequential block scan	O(B * n)	O(n)	Iterates over B blocks; reconstructs Merkle trees of size n per block.
Product Provenance Query	Chronological ledger scan	O(B * T)	O(k)	Filters all T transactions in B blocks; returns k matching history hops.

4.2 Mathematical Proof of O(log n) SPV Verification

Consider a block containing n committed transactions. In a naive auditing system, verifying that transaction Tx is present requires downloading and checking all n transactions, requiring O(n) network bandwidth and O(n) compute time.

In our Merkle Tree implementation, the height h of a binary tree with n leaf nodes is mathematically bounded by:

$$h = \text{ceil}(\log_2(n))$$

At each level from the target leaf to the root, exactly one sibling hash is required to compute the parent hash. Therefore, the total number of cryptographic hashes transmitted in a proof is:

$$|\text{Proof}| = \text{ceil}(\log_2(n))$$

Concrete Scaling Comparison:

- For a block with **1,000 transactions**: log2(1000) ~ 10 hashes. Rather than sending 1,000 transactions (~500 KB), the proof transmits only 10 hashes (320 bytes), achieving a **99.93% bandwidth reduction**.
- For a block with **1,000,000 transactions**: log2(1,000,000) ~ 20 hashes (640 bytes). The verification remains sub-millisecond on mobile consumer devices.

5. Presentation Deck Blueprint (12 Ready Slides for Defense)

This slide-by-slide structure is tailored for capstone evaluations, project defenses, and seminar presentations:

SLIDE 1: TITLE & INTRODUCTION

Blockchain-Based Smart Supply Chain Provenance System

- **Project Goal:** Eliminating counterfeit goods via cryptographic ledgers & Merkle Tree verification.
- **Scope:** Industrial-grade Java implementation using pure standard library & DSA foundations.
- **Key Highlight:** O(log n) lightweight consumer authenticity verification without downloading full chains.

> *Speaker Notes: Welcome evaluators. Today we present our DSA-2 capstone project addressing the global counterfeit crisis through cryptographic data structures.*

SLIDE 2: PROBLEM STATEMENT & INDUSTRIAL MOTIVATION

Why Traditional Supply Chains Fail

- Centralized databases suffer from single-point-of-failure and unauthorized administrative edits.
- Competitors and multi-tier suppliers refuse mutual database write access, creating blind spots.
- Counterfeit injection occurs precisely during custody handoffs between distinct legal entities.

> *Speaker Notes: Point out that SQL databases allow DBAs to update records silently. In our model, math guarantees history cannot be rewritten.*

SLIDE 3: ARCHITECTURAL ARCHITECTURE & DESIGN PRINCIPLES

Clean 4-Tier Layered Architecture

- Strict unidirectional flow: Presentation -> Service -> Ledger/DSA -> Crypto.
- Pluggable Strategy pattern for cryptographic hashing via `HashFunction` interface.
- Immutable Value Objects (`Transaction` , `Block`) prevent state mutation bugs.

> *Speaker Notes: Emphasize our clean separation of concerns and dependency inversion. Outer web layers know about inner logic, never vice versa.*

SLIDE 4: CRYPTOGRAPHIC ENGINE & DETERMINISM

Phase 1: SHA-256 Hashing & Canonical Serialization

- `Sha256Hasher` : Converts UTF-8 strings to 64-character lowercase hex fingerprints.
- Avalanche effect verification: 1-character payload change causes ~50% bit flip divergence.
- `TransactionSerializer` : Deterministic sorted-key JSON serialization via recursive `TreeMap` .

> *Speaker Notes: Explain why serialization determinism is essential: unordered JSON keys would break hash reproducibility and invalidate valid blocks.*

SLIDE 5: THE DSA CORE: MERKLE HASH TREES

Phase 2: Binary Hash Trees & Odd-Leaf Handling

- Bottom-up level-by-level tree generation in O(n) time.
- Odd trailing node pairing rule: duplicates solitary node with itself to preserve balance.
- Tree depth calculation: O(1) depth checks via recursive subtree height tracking.

> *Speaker Notes: Illustrate the binary pairing diagram. Explain how pairing an odd trailing node with itself ensures the tree can process any arbitrary batch size.*

SLIDE 6: LOGARITHMIC VERIFICATION & SPV PROOFS

O(log n) Verification for Mobile Consumer Clients

- DFS recursive path collection gathers sibling hashes with exact directional orientation (LEFT/RIGHT).
- Verification algorithm iterates log2 n times to reconstruct root: O(log n) time, O(1) space.
- Compresses 1,000 transactions down to 10 hashes (99.9% bandwidth savings).

> *Speaker Notes: Highlight this as our primary DSA achievement. A smartphone can verify a drug's authenticity in under 1 millisecond using just 10 hashes.*

SLIDE 7: THE BLOCKCHAIN LEDGER

Phase 3: Cryptographic Linking & Block Genesis

- Genesis Block #0: Anchored with 64 zero characters; establishes system root transaction.
- Block chaining: Block K embeds hash of Block K-1, establishing tamper evidence.
- Global integrity audit: `isValid()` verifies block hashes, link continuity, and Merkle roots.

> *Speaker Notes: Walk through the three integrity checks executed by isValid(). Any single modification breaks both the block's own hash and the next block's link.*

SLIDE 8: SUPPLY CHAIN BUSINESS LOGIC

Phase 4: Provenance Pipeline & Counterfeit Detection

- Root of trust: Manufacturer registration whitelist prevents unauthorized product generation.
- Stateful Aggregate: `Product` tracks current owner and transitions to `SOLD` upon consumer sale.
- 4-stage verification cascade: Registry -> Ledger integrity -> History -> Valid origin.

> *Speaker Notes: Explain how fake products are caught: either they do not exist in registry, or their origin transaction was not initialized by SYSTEM from a whitelisted factory.*

SLIDE 9: LIVE TAMPER DETECTION DEMO

Demonstrating Mathematical Immutability

- `simulateTampering()` modifies an internal transaction's location string.
- Block's calculated hash immediately diverges; Merkle tree recalculation does not match Merkle root.
- Web UI dashboard dynamically displays red alert; `restoreChain()` recovers pristine state.

> *Speaker Notes: If doing a live demo, show the web UI turning red when tamper is clicked, then restored to green. Evaluators love seeing active tamper rejection.*

SLIDE 10: EMPIRICAL BENCHMARK ANALYSIS

Phase 5: Blockchain vs SQLite Relational DB (10k tx)

- SQL is ~20x faster on indexed queries (O(log n) B-Trees vs O(N) linear blockchain scan).
- Blockchain uses ~4x more storage due to hash headers, Merkle nodes, and provenance metadata.
- Blockchain uniquely provides O(log n) cryptographic proof of inclusion and decentralized trust.

> *Speaker Notes: Be academically honest. Blockchain is slower and heavier than SQL. Its justification is not speed, but trustless verification and tamper evidence.*

SLIDE 11: REAL-WORLD CONSTRAINTS & THE ORACLE PROBLEM

Limitations & Enterprise Hybrid Architecture

- **The Oracle Problem:** Blockchains secure digital records, not physical truth.
- Physical security requires PUF tags, encrypted RFID microchips, and sensor feeds.
- **Best-Practice Hybrid Model:** SQL for high-frequency internal ops + Blockchain for trust anchors.

> *Speaker Notes: Discussing the Oracle Problem demonstrates senior engineering maturity. Blockchains cannot prevent someone putting a real QR code on a fake box.*

SLIDE 12: CONCLUSION & TECHNICAL DELIVERABLES

Summary of Capstone Achievements

- 36 production Java classes, ~3,900 lines of code, zero heavy third-party framework bloat.
- Dual testing harness: 100% pass rate across self-test runner and JUnit 5 CI suite.
- Embedded HTTP server hosting full REST API and live presentation dashboard on port 8080.

> *Speaker Notes: Conclude presentation and open floor for viva voce questioning.*

6. Academic Defense & Viva Voce Q&A Guide

Anticipated technical questions from professors, lab examiners, and evaluators, paired with model answers based directly on our implementation:

Q1: Why did you use a Merkle Tree instead of just hashing the concatenated list of transactions?

Model Answer: If we merely concatenated all transactions and hashed them once, verifying that a single transaction exists inside a block would require downloading all N transactions, yielding an O(N) network and compute burden. With a Merkle Tree, a client only needs ceil(log2 N) sibling hashes. For a block with 1,000 transactions, verification requires only 10 hashes (~320 bytes), allowing lightweight consumer mobile apps (SPV mode) to verify authenticity instantaneously.

Q2: How does your implementation handle an odd number of transactions in a block during Merkle tree construction?

Model Answer: In `MerkleTree.buildTree()`, we check adjacent pairs: `(i, i+1)`. If `i + 1 >= currentLevel.size()`, meaning a solitary node exists at the end of the level, we duplicate that solitary node and hash it with itself: `right = left`. This preserves complete binary balance at every level without corrupting mathematical determinism.

Q3: What makes your transaction hashing deterministic across different JVMs and platforms?

Model Answer: Standard JSON formatters do not guarantee key order. If one JVM produces `{"location":"A","sender":"B"}` and another produces `{"sender":"B","location":"A"}`, the SHA-256 hashes would differ. In `TransactionSerializer.java`, we recursively load all map keys into sorted `java.util.TreeMap` structures before string construction, guaranteeing byte-for-byte identical UTF-8 serializations regardless of runtime environment.

Q4: What is the time complexity of validating the blockchain, and can it be optimized?

Model Answer: Global chain validation in `Blockchain.isValid()` is O(B * N), where B is the number of blocks and N is the number of transactions per block, because it recomputes every block hash and Merkle root from scratch. In production systems, this is optimized by caching validated block headers and using Merkle Mountain Ranges (MMR) or cryptographic accumulators, validating only newly appended blocks in O(1) amortized time.

Q5: What is the "Oracle Problem" in supply chain blockchains, and how does your project address it?

Model Answer: The Oracle Problem is the gap between digital data and physical reality. The blockchain immutably verifies digital transaction records, but cannot physically guarantee that the contents of a shipping container match the digital label. Our project documents this trade-off explicitly and incorporates whitelisted manufacturer origin checks (`SYSTEM -> Factory`) and recommends cryptographic PUF (Physical Unclonable Functions) and sensor integrations as real-world bridges.

Q6: Why did SQLite perform faster on queries than the blockchain during your benchmarks?

Model Answer: SQLite utilizes balanced B-Trees over indexed columns (`CREATE INDEX idx_product_id`), giving O(log M) record retrieval where M is total rows. Our basic in-memory blockchain queries history via linear traversal across all blocks and transactions (O(B * T)). While adding an in-memory HashMap index to the blockchain resolves read speeds, the relational database natively excels at arbitrary querying and joins.

Q7: How did you implement tamper detection without corrupting memory during live demos?

Model Answer: `Blockchain.java` includes an educational test hook: `simulateTampering()`. Before modifying a target block's location payload, it saves a snapshot in `backupChain`. When `isValid()` runs, Check 3 fails because the recomputed Merkle root of the modified transaction does not match the block's immutable stored `merkleRoot`. The method `restoreChain()` restores the snapshot, allowing repeatable live testing.

Q8: Why is directionality (LEFT vs RIGHT) necessary in Merkle Proofs?

Model Answer: The SHA-256 hash function is non-commutative: H(A + B) != H(B + A). In `MerkleProofElement.java`, we store a `MerkleSide` enum. During verification in `MerkleTree.verifyProof()`, if the sibling is marked `LEFT`, we compute H(sibling + current); if marked `RIGHT`, we compute H(current + sibling). Without side orientation, recomputing the root hash would be impossible.

7. Conclusion & Production Roadmap

7.1 Project Summary & Achievements

This capstone project successfully demonstrates that complex cryptographic and distributed ledger principles can be engineered with extreme clarity and performance using foundational Data Structures and Algorithms in Java. The project delivers:

- A mathematically verified SHA-256 cryptographic hashing core with zero external library overhead.
- A complete binary Merkle Tree engine achieving true O(log n) membership proofs.
- A clean, immutable chained block ledger with full tamper detection.
- A realistic multi-stage supply chain lifecycle tracking products from factory birth to consumer retail sale.
- An objective, empirical benchmark against relational databases highlighting practical engineering trade-offs.
- A zero-dependency embedded presentation web dashboard for live demonstrations.

7.2 Future Production Roadmap

To evolve this academic solution into an enterprise-grade global supply chain network, the following enhancements are outlined:

1. **Consensus Layer:** Implementing Byzantine Fault Tolerant (PBFT) or Raft consensus across distributed network nodes to replace the current single-node sequencer.
2. **Zero-Knowledge Proofs (zk-SNARKs):** Allowing suppliers to prove compliance with regulatory temperature and customs thresholds without exposing confidential pricing or supplier identities.
3. **Smart Contract Automation:** Writing programmable logic for automated escrow release, insurance payouts on sensor threshold breach, and automated customs clearance.
4. **Hardware IoT Anchoring:** Direct integration with LoRaWAN and RFID sensors writing cryptographically signed telemetry directly into transaction metadata blocks.

— End of Technical Specification & Full Solution Report —